Documentación Técnica: Algoritmo de Limpieza de Inasistencias mediante FSM

Desde la Especificación Matemática Formal hasta la Implementación de Nivel de Producción y Pruebas Unitarias

Dominio: Limpieza ETL de Reportes Corruptos	**Modelo:** Máquina de Estados Finitos (FSM)
Tolerancia: Invariante de Desplazamiento Dynamic Phase-Shift	**Lenguaje / Testing:** Python 3 (Standard Library + unittest)

1. Especificación Matemática y Lógica Formal

El problema de limpieza de datos en el reporte de inasistencias se fundamenta en un modelo abstracto independiente del contexto de implementación o del número de fila. El objetivo es corregir el error de desplazamiento de columnas (asociado históricamente a filas como las comprendidas entre 7485 y 16674) mediante propiedades semánticas y lógica de predicados.

1.1 Espacio de Datos y Dominio Semántico

- **Matriz de Entrada (M):** Sea $M \in C^{N*K}$ una matriz bidimensional no homogénea de N filas y K columnas, donde cada celda $C_{i,j} \in C$ pertenece al universo de valores (cadenas, números o nulo ∅).
- **Vector Fila (r_i):** Cada fila se define como la tupla $\boldsymbol{r_i} = (c_{i,0}, c_{i,1}, \dots c_{i,K-1},)$ para $i \in \{0, 1, \dots, N-1\}$
- **Predicados de Tipo Semántico (P_t):** Familia de funciones indicadoras de tipo $\boldsymbol{P_t}: \boldsymbol{C} \rightarrow \{\mathbf{0}, \mathbf{1}\}$:
  - $\boldsymbol{P_{ID}}(\boldsymbol{c})$: Valida si c satisface la estructura de un identificador único (ej. Retorna 1 si c cumple el patrón de un identificador numérico/alfanumérico ($4 \leq longitud \leq 12$))
  - $\boldsymbol{P_{Texto}}(\boldsymbol{c})$: Valida si c es una cadena alfabética (ej. Retorna 1 si c es una cadena alfabética válida.).
  - $\boldsymbol{P_{Fecha}}(\boldsymbol{c})$: Valida si c cumple con una marca temporal o fecha válida (ej. Retorna 1 si c cumple con la estructura ISO o de marca temporal (AAAA-MM-DD)).
  - $\boldsymbol{P_{Booleano}}(\boldsymbol{c})$: Valida si c representa un estado binario (Justificada / No Justificada). Ej. Retorna 1 si c representa un estado de justificación.
  - $\boldsymbol{P_{\emptyset}}(\boldsymbol{c})$: Retorna 1 si $c = \emptyset$ (celda nula o vacía).
- **Esquema Semántico Objetivo (T):** La entidad relacional destino se define como la tupla ordenada de tipos esperados:

$$T = (t_0, t_1, t_2, t_3, t_4) = (P_{ID}, P_{Texto}, P_{Fecha}, P_{Texto}, P_{Booleano})$$

1.2 Lógica de Filtrado de Encabezados y Metadatos Parásitos

- **Firma de Encabezado (S_{meta}):** conjunto de Conjunto de palabras clave asociadas a títulos e impresiones de paginación

$(S_{meta}$
$= \{$"Instructor", "Ficha", "Aprendiz", "$Identificación\ Aprendiz$", "$Fecha\ Inicio$", "$Fecha\ Fin$", "$Cant.horas$", "$Justificación$"$\}$)

- Predicado de Invariante de Registro $(EsRegistroValido(r_i))$:
  $EsRegistroValido(r_i) \Leftrightarrow \exists j \in \{0, \dots, K-1\}$ tal que $\boldsymbol{P_{ID}}(c_{i,j}) = 1 \wedge \forall c \in r_i, c \notin S_{meta}$

1.3 Modelo Dinámico de Detección y Corrección de Desplazamiento (Phase-Shift)

En lugar de evaluar la posición absoluta de la fila, la alineación se calcula determinando la desviación de fase horizontal (δ_i) de la firma de tipos de la fila respecto al Esquema Objetivo T:

- **Vector de Firma Observado** $F(r_i)$**:** $F(r_i) = \left(P(c_{i,0}), P(c_{i,1}), \dots, P(c_{i,K-1})\right)$
- **Función de Ajuste Semántico:** Para un desplazamiento $d \in \mathbb{Z} \geq 0$, la compatibilidad entre la fila desplazada y Tes:

$$Ajuste(r_i, d) = \sum_{j=0}^{m-1} \mathbb{I}\left(P_{t_j}(c_{i,j+d}) = 1\right)$$

En donde $\mathbb{I}$ es la función indicadora y $d \in \mathbb{Z} \geq 0$ representa el desplazamiento probado.

- **Cálculo de la Desviación Óptima** (δ_i)**:**

$$\delta_i = arg \max_{d \in \{0,\dots,K-1\}} Ajuste(r_i, d)$$

Si la fila está alineada, $\delta_i = 0$. Si comenzó una casilla a la derecha (debido a celda nula en la posición 0), $\delta_i = 1$ de forma automática sin depender del índice i.

- **Operador de Realineación** $(\boldsymbol{\tau_\delta})$**:**

$$\tau_\delta(r_i) = (c_{i,j+d})_{j=0}^{m-1}$$

1.3 Operadores de Descomposición Atómica y Proyección Relacional

Operador de Fisión (α): Separa celdas compuestas (ej. "Apellidos, Nombres") unidas por un delimitador $\oplus =$ ",":

📌 Formalismo de Fisión y Transformación Total

$$\alpha(c, \oplus) = \begin{cases} (a, b)\ si\ c = a \oplus b \\ (c, \emptyset) en\ otro\ caso \end{cases}$$

Transformación Total del Conjunto de Datos $(\mathcal{L})$:

$$\mathcal{L} = \{\pi(c'_{i,0}, \alpha(c'_{i,1}, ","), c'_{i,2}, c'_{i,3}, c'_{i,4}) | r_i \in M \wedge EsRegistroVálido(r_i)\}$$

Donde $c'_{i,j}$ son celdas de la tupla realineada $\tau_\delta(r_i)$ y π es el operador de proyección.

2. Diseño de la Máquina de Estados Finitos (FSM)

La Máquina de Estados Finitos (FSM) actúa como el controlador en flujo (stream-driven) que procesa el archivo fila por fila, evaluando eventos y cambiando de estado para garantizar un procesamiento robusto e inmune a fallos.

El autómata determinista se formaliza como la 5-tupla

$$M = (Q, \sum, \delta_{fsm}, q0, F)$$

2.1 Componentes del Autómata

$Estados\ (Q)$: Q
$= \{\text{INIT}, \text{SCAN}_{\text{ROW}}, \text{IGNORE}_{\text{NOISE}}, \text{REALIGN}, \text{SPLIT}_{\text{FIELDS}}, \text{EMIT}_{\text{RECORD}}, \text{ERROR}_{\text{RECOVERY}}, \text{FINISH}\}$

Estado inicial (q_0): INIT
Estado final (F): FINISH

Alfabeto de eventos ($\sum$): E_{meta} (línea de título/vacía), E_{fase0} (fila alineada), $\text{E}_{\text{desfasada}}$(fila desplazada $(\delta_i > 0)$), $\text{E}_{\text{inválida}}$, E_{EOF} fin de archivo)

Tabla 2.1: Matriz de Transiciones de la FSM (δ: Q × Σ → Q)

Estado Actual (Q)	Evento Entrada (Σ)	Estado Siguiente (Q')	Acción Ejecutada
INIT	Inicio Ejecución	SCAN_{ROW}	Carga esquema objetivo T
SCAN_{ROW}	E_meta / E_vacía	$\text{IGNORE}_{\text{NOISE}}$	Identifica fila no deseada
$\text{IGNORE}_{\text{NOISE}}$	Fila descartada	SCAN_{ROW}	Salta a la siguiente fila
SCAN_{ROW}	E_{fase0}/ $\text{E}_{\text{desfasada}}$	REALIGN	Evalúa alineamiento
REALIGN	Calculado δ_i	$\text{SPLIT}_{\text{FIELDS}}$	Aplica realineación τ_δ
$\text{SPLIT}_{\text{FIELDS}}$	Fisión α lista	$\text{EMIT}_{\text{RECORD}}$	Divide campos compuestos
$\text{EMIT}_{\text{RECORD}}$	Registro emitido	SCAN_{ROW}	Escribe tupla y pide datos

$SCAN_{ROW}$	$E_{inválida}$	$ERROR_{RECOVERY}$	Registra fallo en log
$ERROR_{RECOVERY}$	Error gestionado	$SCAN_{ROW}$	Continúa sin detener el programa
$SCAN_{ROW}$	E_{EOF}	FINISH	Cierra flujos y finaliza

2.2 Invariantes de la Máquina de Estados

- **Invariante de Memoria Acotada O(1):** La FSM solo mantiene en memoria la fila actual procesada (r_i) y las variables de estado. Nunca carga el conjunto masivo de datos en memoria simultáneamente.
- **Invariante de Aislamiento de Errores:** Un evento $E_{inválida}$ transiciona a $ERROR_{RECOVERY}$ y jamás causa la caída colateral (crash) del hilo principal de procesamiento.
- **Invariante de Salida Estandarizada:** Todo registro emitido en EMIT_RECORD cumple strictly con las dimensiones y tipos declarados en el Esquema Objetivo T.

3. Pseudocódigo Estructurado de la FSM

Traducción de la FSM mediante el patrón Event-Driven con estructura Bucle MIENTRAS + SEGÚN (Switch/Case):

Pseudocódigo Estructurado Event-Driven FSM

```
ALGORITMO LimpiarInasistenciasFSM(stream_entrada, stream_salida):

    // 1. DEFINICIÓN DE ESTADOS
    ESTADOS = {INIT, SCAN_ROW, IGNORE_NOISE, REALIGN, SPLIT_FIELDS, EMIT_RECORD,
ERROR_RECOVERY, FINISH}

    // 2. VARIABLES DE CONTROL Y MEMORIA (O(1))
    estado_actual = INIT
    esquema_objetivo T = [EsID, EsTexto, EsFecha, EsTexto, EsBooleano]

    fila_raw = NULO
    fila_alineada = NULO
    registro_limpio = NULO
    delta_i = 0

    // 3. BUCLE PRINCIPAL DE LA FSM
    MIENTRAS estado_actual != FINISH HACER:

        SEGÚN estado_actual HACER:

            CASO INIT:
                CargarEstructuraValida(T)
                estado_actual = SCAN_ROW
```

```
            CASO SCAN_ROW:
                SI HayMasFilas(stream_entrada) ENTONCES:
                    fila_raw = LeerSiguienteFila(stream_entrada)

                    // Evaluación de Eventos de Entrada
                    SI EsFilaVacia(fila_raw) O EsEncabezadoRepetido(fila_raw) ENTONCES:
                        estado_actual = IGNORE_NOISE
                    SINO SI EsEstructuraValida(fila_raw) ENTONCES:
                        estado_actual = REALIGN
                    SINO:
                        estado_actual = ERROR_RECOVERY
                SINO:
                    estado_actual = FINISH // Fin de archivo (EOF)

            CASO IGNORE_NOISE:
                RegistrarLog("Fila parásita o vacía ignorada")
                estado_actual = SCAN_ROW

            CASO REALIGN:
                delta_i = CalcularDesfaseOptimo(fila_raw, T)

                SI delta_i != ERROR_ALIGNMENT ENTONCES:
                    fila_alineada = DesplazarHorizontalmente(fila_raw, delta_i)
                    estado_actual = SPLIT_FIELDS
                SINO:
                    estado_actual = ERROR_RECOVERY

            CASO SPLIT_FIELDS:
                nombre_compuesto = fila_alineada[COL_NOMBRE]
                apellidos, nombres = SepararPorDelimitador(nombre_compuesto,
delimitador=",")

                registro_limpio = ConstruirTupla(fila_alineada[ID], apellidos, nombres,
fila_alineada[FECHA], fila_alineada[ESTADO])
                estado_actual = EMIT_RECORD

            CASO EMIT_RECORD:
                EscribirRegistro(stream_salida, registro_limpio)
                estado_actual = SCAN_ROW

            CASO ERROR_RECOVERY:
                RegistrarErrorEnLog("Fila highly corrupta que no coincide con T",
fila_raw)
                estado_actual = SCAN_ROW

        FIN SEGÚN

        RETORNAR "Procesamiento FSM Finalizado Exitosamente"
FIN ALGORITMO
```

4. Suite de Pruebas Unitarias (unittest)

A continuación se incluye la suite de pruebas unitarias implementada con la biblioteca estándar `unittest` de Python para aislar y validar cada estado de la FSM ante escenarios normales y anómalos:

📌 **test_fsm_inasistencias.py (Pruebas Unitarias)**

```
import unittest

class FSMInasistencias:
    def __init__(self):
        self.estado = 'INIT'
        self.registros_emitidos = []
        self.log_errores = []

    def es_encabezado_o_vacia(self, fila):
        if not fila or all(str(c).strip() == '' for c in fila):
            return True
        primer_elem = str(fila[0]).upper().strip()
        return primer_elem in ['ID', 'DOCUMENTO', 'ESTUDIANTE', 'REPORTE']

    def calcular_desfase_optimo(self, fila):
        if len(fila) > 1 and (fila[0] == '' or fila[0] is None) and
str(fila[1]).isdigit():
            return 1
        if len(fila) > 0 and str(fila[0]).isdigit():
            return 0
        return -1

    def separar_nombres(self, texto_nombre):
        if ',' in str(texto_nombre):
            partes = str(texto_nombre).split(',', 1)
            return partes[0].strip(), partes[1].strip()
        return str(texto_nombre).strip(), ''

    def procesar_siguiente(self, fila_raw):
        if self.es_encabezado_o_vacia(fila_raw):
            self.estado = 'IGNORE_NOISE'
            return None

        delta = self.calcular_desfase_optimo(fila_raw)
        if delta == -1:
            self.estado = 'ERROR_RECOVERY'
            self.log_errores.append(f"Fila no coincide con el esquema: {fila_raw}")
            return None

        self.estado = 'REALIGN'
        fila_alineada = fila_raw[delta:]
```

```
        self.estado = 'SPLIT_FIELDS'
        apellidos, nombres = self.separar_nombres(fila_alineada[1])

        self.estado = 'EMIT_RECORD'
        registro = {
            'id': fila_alineada[0],
            'apellidos': apellidos,
            'nombres': nombres,
            'fecha': fila_alineada[2] if len(fila_alineada) > 2 else '',
            'estado': fila_alineada[3] if len(fila_alineada) > 3 else ''
        }
        self.registros_emitidos.append(registro)
        return registro


class TestFSMInasistencias(unittest.TestCase):

    def setUp(self):
        self.fsm = FSMInasistencias()

    def test_01_fila_normal_alineada(self):
        fila_normal = ['1001', 'Pérez, Juan', '2026-10-01', 'Justificada']
        resultado = self.fsm.procesar_siguiente(fila_normal)
        self.assertEqual(self.fsm.estado, 'EMIT_RECORD')
        self.assertIsNotNone(resultado)
        self.assertEqual(resultado['id'], '1001')
        self.assertEqual(resultado['apellidos'], 'Pérez')
        self.assertEqual(resultado['nombres'], 'Juan')

    def test_02_fila_desplazada_shift_1(self):
        fila_corrupta = ['', '1002', 'Gómez, María', '2026-10-02', 'Injustificada']
        resultado = self.fsm.procesar_siguiente(fila_corrupta)
        self.assertEqual(self.fsm.estado, 'EMIT_RECORD')
        self.assertEqual(resultado['id'], '1002')
        self.assertEqual(resultado['apellidos'], 'Gómez')
        self.assertEqual(resultado['nombres'], 'María')

    def test_03_descarte_de_encabezados_y_ruido(self):
        fila_encabezado = ['ID', 'Estudiante', 'Fecha', 'Estado']
        resultado = self.fsm.procesar_siguiente(fila_encabezado)
        self.assertEqual(self.fsm.estado, 'IGNORE_NOISE')
        self.assertIsNone(resultado)
        self.assertEqual(len(self.fsm.registros_emitidos), 0)

    def test_04_fision_nombres_compuestos(self):
        apellidos, nombres = self.fsm.separar_nombres("Rodríguez Silva, Ana Sofía")
        self.assertEqual(apellidos, "Rodríguez Silva")
        self.assertEqual(nombres, "Ana Sofía")
```

```
    def test_05_recuperacion_de_errores(self):
        fila_basura = ['TextoInvalido', 'SinNumeros', '###']
        resultado = self.fsm.procesar_siguiente(fila_basura)
        self.assertEqual(self.fsm.estado, 'ERROR_RECOVERY')
        self.assertIsNone(resultado)
        self.assertEqual(len(self.fsm.log_errores), 1)

if __name__ == '__main__':
    unittest.main()
```

5. Código Ejecutable en Python de Nivel de Producción

Código Python completo para producción con encapsulamiento orientado a objetos y manejo de expresiones regulares para la detección de tipos:

fsm_inasistencias_cleaner.py (Código Producción Python)

```
from enum import Enum, auto
import pandas as pd
import re
import sys

class EstadoFSM(Enum):
    INIT = auto()
    SCAN_ROW = auto()
    IGNORE_NOISE = auto()
    REALIGN = auto()
    SPLIT_FIELDS = auto()
    EMIT_RECORD = auto()
    ERROR_RECOVERY = auto()
    FINISH = auto()

class LimpiadorInasistenciasFSM:
    def __init__(self, archivo_entrada: str, archivo_salida: str):
        self.archivo_entrada = archivo_entrada
        self.archivo_salida = archivo_salida
        self.estado_actual = EstadoFSM.INIT
        self.registros_procesados = []
        self.log_errores = []
        self.patron_meta = re.compile(r'^(ID|DOCUMENTO|ESTUDIANTE|REPORTE|FECHA|PÁGINA)',
re.IGNORECASE)

    def es_celda_vacia(self, val) -> bool:
        return pd.isna(val) or str(val).strip() == '' or str(val).strip().lower() == 'nan'

    def es_id_valido(self, val) -> bool:
        val_str = str(val).strip()
        return bool(re.match(r'^\d{4,12}$', val_str))
```

```
    def es_ruido_o_cabecera(self, fila: list) -> bool:
        if not fila or all(self.es_celda_vacia(c) for c in fila):
            return True
        primer_contenido = next((str(c).strip() for c in fila if not
self.es_celda_vacia(c)), '')
        return bool(self.patron_meta.search(primer_contenido))

    def calcular_desfase_optimo(self, fila: list) -> int:
        for idx, celda in enumerate(fila[:5]):
            if self.es_id_valido(celda):
                return idx
        return -1

    def separar_apellidos_nombres(self, texto_compuesto: str) -> tuple:
        texto = str(texto_compuesto).strip()
        if ',' in texto:
            partes = texto.split(',', 1)
            return partes[0].strip(), partes[1].strip()
        return texto, ''

    def ejecutar(self):
        print("[FSM] Iniciando máquina de estados...")
        self.estado_actual = EstadoFSM.SCAN_ROW
        df_raw = pd.read_excel(self.archivo_entrada, header=None)

        for idx, fila_series in df_raw.iterrows():
            fila = fila_series.tolist()

            if self.estado_actual == EstadoFSM.SCAN_ROW:
                if self.es_ruido_o_cabecera(fila):
                    self.estado_actual = EstadoFSM.IGNORE_NOISE
                else:
                    self.estado_actual = EstadoFSM.REALIGN

            if self.estado_actual == EstadoFSM.IGNORE_NOISE:
                self.estado_actual = EstadoFSM.SCAN_ROW
                continue

            if self.estado_actual == EstadoFSM.REALIGN:
                delta = self.calcular_desfase_optimo(fila)
                if delta != -1:
                    fila_alineada = fila[delta:]
                    self.estado_actual = EstadoFSM.SPLIT_FIELDS
                else:
                    self.estado_actual = EstadoFSM.ERROR_RECOVERY

            if self.estado_actual == EstadoFSM.ERROR_RECOVERY:
                self.log_errores.append({'fila_index': idx, 'contenido': fila})
                self.estado_actual = EstadoFSM.SCAN_ROW
```

```
                continue

            if self.estado_actual == EstadoFSM.SPLIT_FIELDS:
                id_estudiante = str(fila_alineada[0]).strip()
                nombre_raw = fila_alineada[1] if len(fila_alineada) > 1 else ''
                fecha = fila_alineada[2] if len(fila_alineada) > 2 else ''
                tipo_inasistencia = fila_alineada[3] if len(fila_alineada) > 3 else ''
                justificada = fila_alineada[4] if len(fila_alineada) > 4 else ''

                apellidos, nombres = self.separar_apellidos_nombres(nombre_raw)
                self.estado_actual = EstadoFSM.EMIT_RECORD

            if self.estado_actual == EstadoFSM.EMIT_RECORD:
                self.registros_procesados.append({
                    'ID_Estudiante': id_estudiante,
                    'Apellidos': apellidos,
                    'Nombres': nombres,
                    'Fecha': fecha,
                    'Tipo_Inasistencia': tipo_inasistencia,
                    'Justificada': justificada
                })
                self.estado_actual = EstadoFSM.SCAN_ROW

        self.estado_actual = EstadoFSM.FINISH
        self._exportar_resultados()

    def _exportar_resultados(self):
        df_limpio = pd.DataFrame(self.registros_procesados)
        df_limpio.to_excel(self.archivo_salida, index=False)
        print(f"[FSM] Proceso finalizado. Registros válidos: {len(df_limpio)} | Filas
anómalas: {len(self.log_errores)}")

if __name__ == '__main__':
    limpiador = LimpiadorInasistenciasFSM("Inasistencias.xlsx",
"Inasistencias_Limpio.xlsx")
    # limpiador.ejecutar()
```